



GYM MANAGEMENT SYSTEM

INDUSTRIAL INFORMATICS PROJECT

Team leader:	Bogdan Dora	Group: 30332/1
Frontend developer:	Salajanu Mircea Vasile	Group: 30136/2
Backend developer:	Simionca Alexandru Petru Florin	Group: 30136/2
Full-Stack developer:	Kovacs Erik	Group: 30136/2
Tester:	Luidort Zsuzsa	Group: 30332/1

MINISTERUL EDUCAȚIEI NAȚIONALE



UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE



PROJECT SUMMARY

of the project titled:

«Gym Management System»

1. Project requirements:

The project required the development of a **complex software application** following **Object-Oriented Programming (OOP)** principles, with a **structured software architecture**, a **graphical user interface (GUI)**, and **persistent data management**.

For this project, the team chose to develop a **Gym Management System**, designed to manage gym members, subscriptions, payments, attendance, and role-based user access through a functional web application.

2. Chosen solutions:

The project was implemented using a **three-tier software architecture**, separating the application into frontend, backend, and database layers.

The **frontend** was developed using **Angular**, providing a graphical user interface with role-based dashboards and interactive pages for users.

The **backend** was implemented using **ASP.NET Core Web API**, responsible for business logic, authentication, CRUD operations, and communication with the frontend through REST API endpoints.

For persistent data management, **SQL Server** and **Entity Framework Core** were used to store and manage application data. Additional tools such as **Swagger** were used for API testing, **Git/GitHub** for version control, and **Figma** for UI design.

3. Results obtained:

A functional version of the **Gym Management System** was successfully developed. The application includes a working login system, role-based authentication, operational dashboards for different user roles, CRUD functionalities for core entities, and database integration.

The system allows management of members, subscriptions, payments, attendance records, and role-based dashboard navigation, demonstrating the main functionalities required for gym management operations.



4. Testing and verification:

Testing and verification were performed using **Swagger** for backend API validation, manual frontend testing, and SQL Server database verification.

The login system, CRUD functionalities, role-based authentication, dashboard redirection, and database persistence were tested to ensure correct application behavior and proper communication between frontend and backend.

Table of Contents

1. CHAPTER 1 – INTRODUCTION.....	2
1.1 PROBLEM STATEMENT	2
1.2 OBJECTIVES	2
1.3 SPECIFICATIONS	2
2. CHAPTER 2 – APPLICATION ARCHITECTURE.....	3-6
1. 2.1. CLASS DIAGRAM	3
2. 2.2. USE CASE DIAGRAM	4
3. 2.3. DATABASE DIAGRAM	5
4. 2.4. ARCITECTURE DIAGRAM	6
3. CHAPTER 3 – APPLICATION IMPLEMENTATION	7
4. CHAPTER 4 – APPLICATION TESTING	7
5. APPENDIX 1 – SOURCE CODE	7

1. Chapter 1 – INTRODUCTION

1.1 Problem Statement

Traditional gym management often relies on manual processes for handling members, subscriptions, payments, and attendance records. This can lead to inefficiency, data inconsistency, and difficulties in organizing gym operations.

The purpose of this project is to provide a digital solution that centralizes gym management activities into a single web-based application with role-based access and secure data management.

1.2 Objectives

Main objectives:

- Develop a functional gym management application;
- Implement role-based authentication and access control;
- Manage gym members and subscriptions;
- Track payments and attendance records;
- Provide dashboards for different user roles.

Secondary objectives:

- Create an intuitive graphical user interface;
- Ensure persistent data storage;
- Enable communication between frontend and backend;
- Verify and test application functionalities;

1.3 Specifications

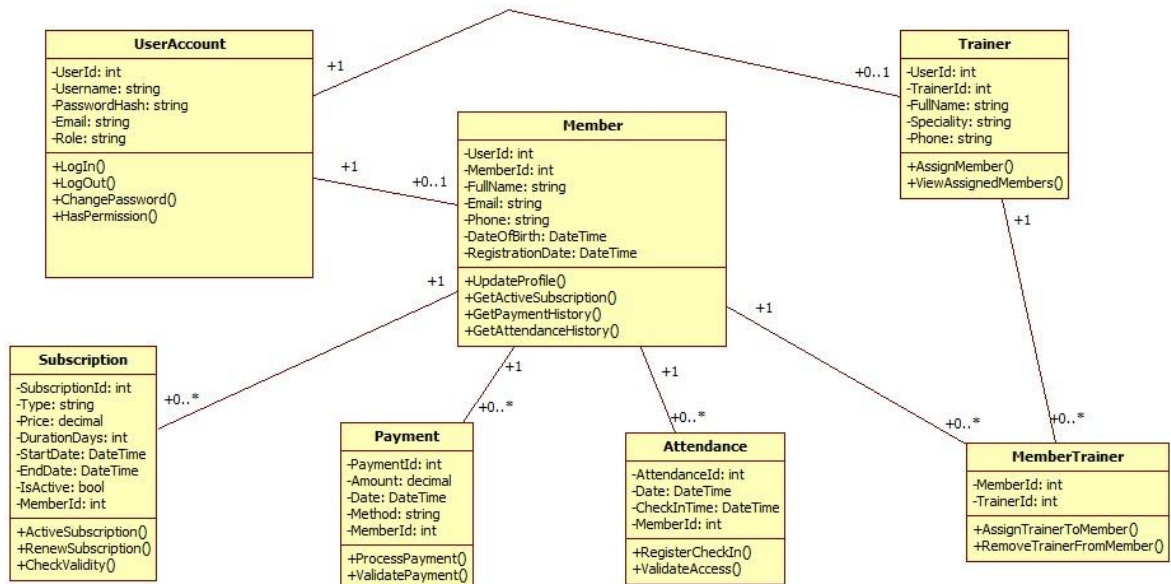
The application was developed using a structured software architecture and modern technologies.

- **Frontend:** Angular
- **Backend:** ASP.NET Core Web API
- **Database:** SQL Server
- **ORM:** Entity Framework Core
- **Programming languages:** TypeScript, C#
- **API testing:** Swagger
- **Version control:** Git/GitHub
- **UI design:** Figma

The system implements role-based authentication, CRUD operations, REST API communication, and persistent relational database management.

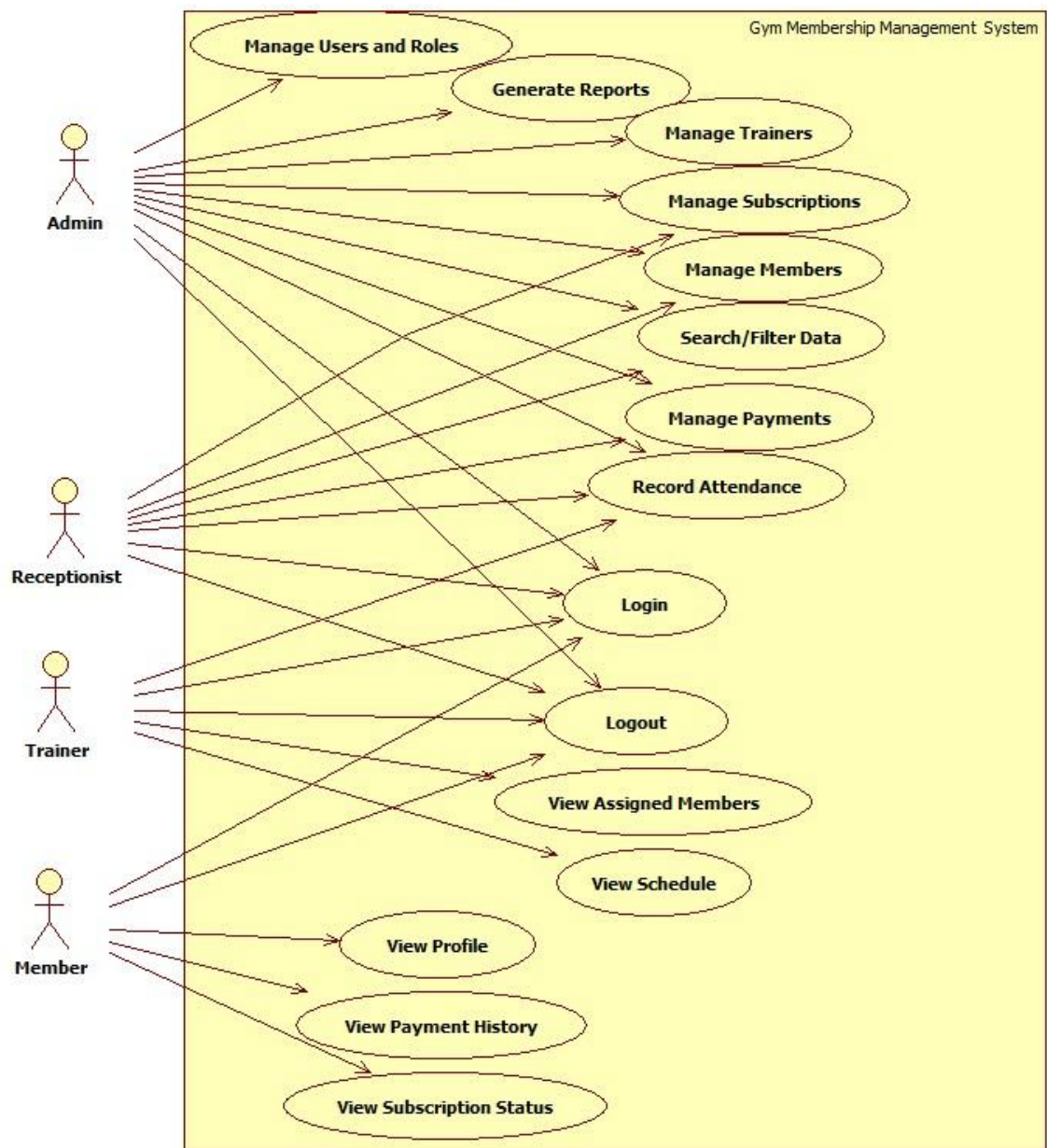
2. Chapter 2 – APPLICATION ARCHITECTURE

1. 2.1. Class Diagram – “What is the system made of?”



The **UML Class Diagram** presents the main classes of the Gym Management System, including their attributes, methods, and relationships. The main functionality is centered around Members, Trainers, Subscriptions, Payments, and Attendance. The system also includes a UserAccount class for login and role management. One-to-many relationships are used for subscriptions, payments, and attendance, while the many-to-many relationship between Members and Trainers is handled through the MemberTrainer class.

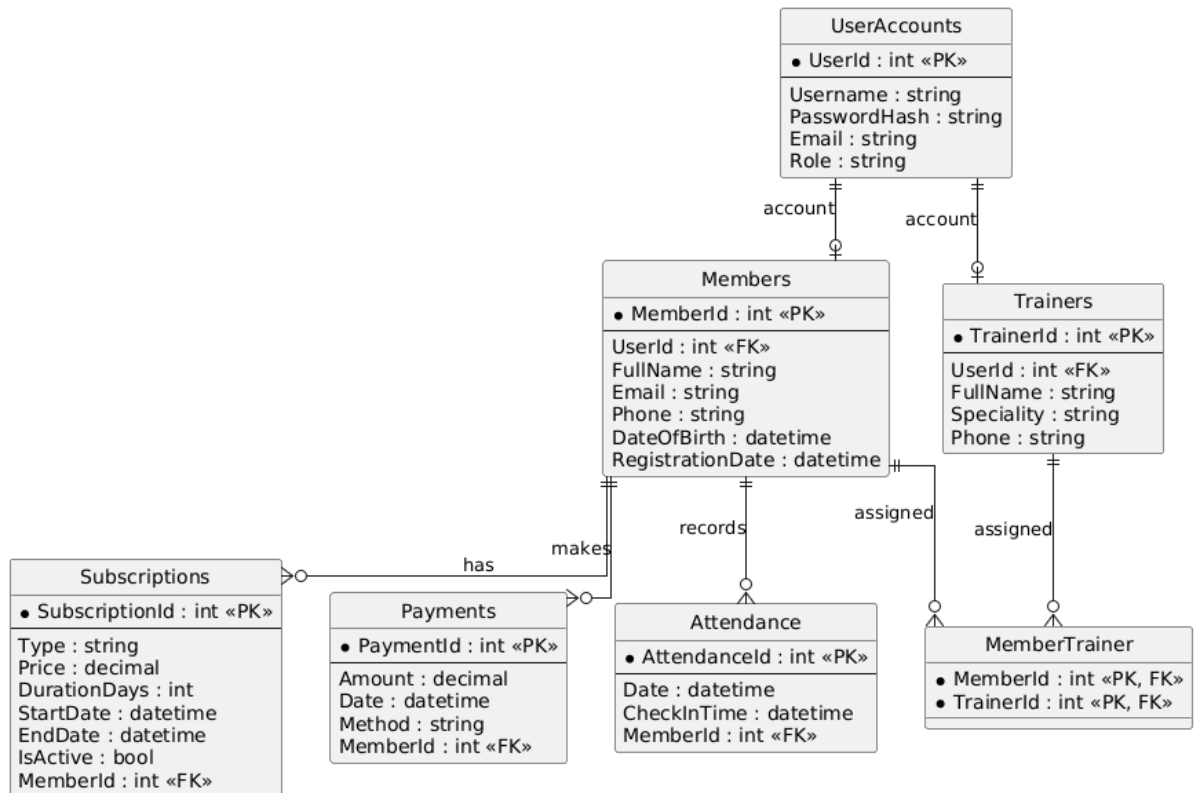
2. 2.2. Use Case Diagram – “What does the system do?”



The **Use Case Diagram** illustrates the interactions between the system actors (Admin, Receptionist, Trainer, Member) and the available system functionalities. Each role has specific permissions and access rights within the Gym Management System.

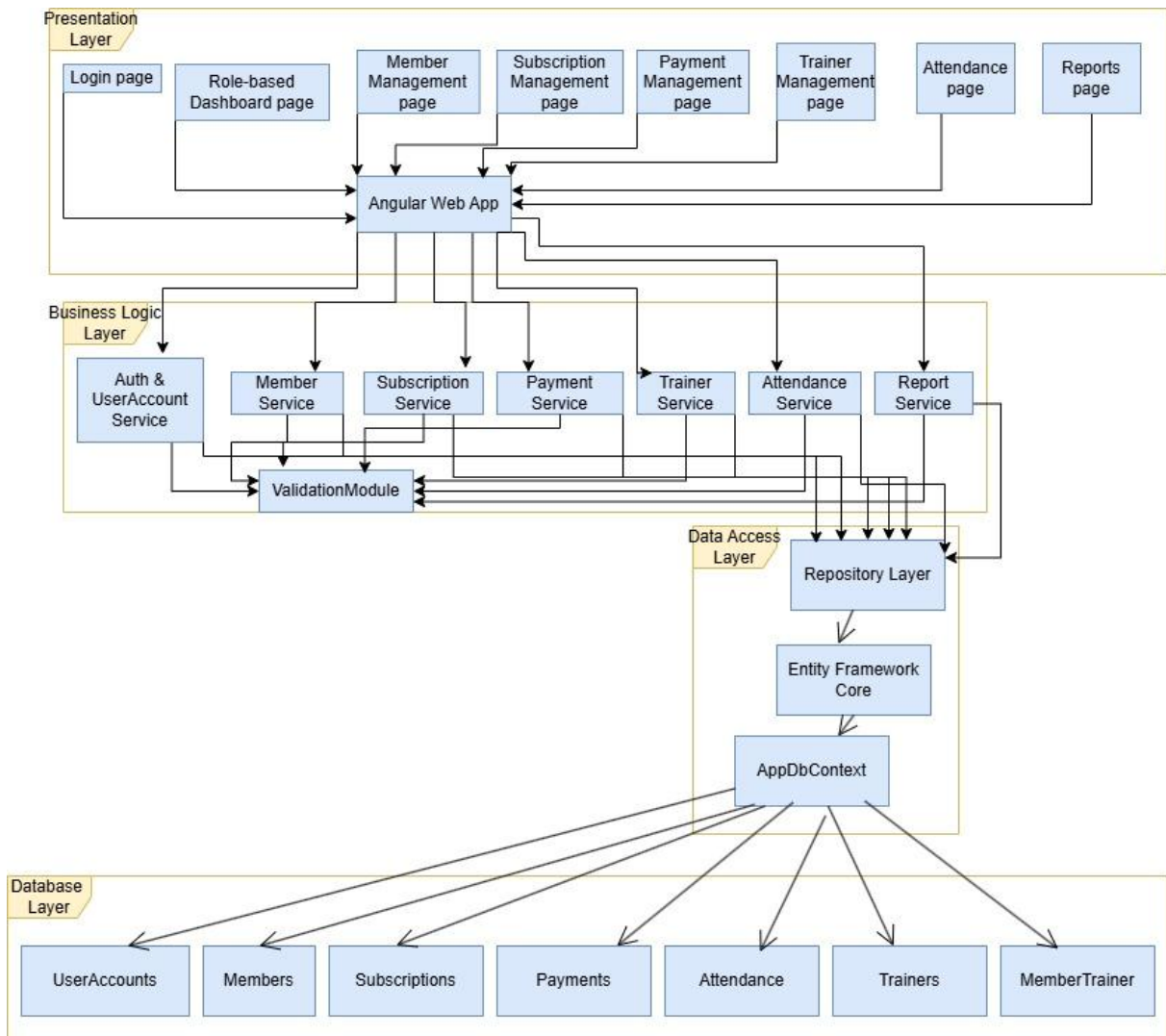
- **Admin:** Manage members, trainers, subscriptions, payments, attendance, reports, and users/roles.
- **Receptionist:** Add and update members, view subscriptions, register payments, and check attendance.
- **Trainer:** View assigned members, view schedule, and record attendance/sessions.
- **Member:** View personal profile, subscription status, and payment history.

3. 2.3. Database Diagram – “How is the data stored?”



The **ERD** shows how the database is organized, including entities, primary keys, foreign keys, and relationships between tables. The **UserAccounts** table is used for authentication and role management. **Members** can have multiple subscriptions, payments, and attendance records, while the relationship between **Members** and **Trainers** is managed through the **MemberTrainer** table.

4. 2.4. Architecture Diagram – “How is the system built?”



The **Architecture Diagram** illustrates the layered structure of the Gym Management System. The Presentation Layer is implemented using Angular, the Business Logic Layer is developed using ASP.NET Core Web API, and the Data Access Layer uses Entity Framework Core and SQL Server. This structure ensures modularity, maintainability, and separation of responsibilities.

3. Chapter 3 – APPLICATION IMPLEMENTATION

This chapter may be divided into **subchapters**, for example:

- database implementation (how it was implemented);
- server-side implementation;
- client-side implementation;
- application login system;
- etc.
- Include **code snippets and explanations**.

4. Chapter 4 – APPLICATION TESTING

- test cases;
- explanation of how the application works;
- errors encountered during development and how they were handled.

5. Appendix 1 – Source Code